

SOC Final Project Report

Created time: August 5, 2024 8:25 PM Reviewed: No

Problem Statement:

Fine-grained classification: Train a CNN model with an upper limit of 10M parameters on the CUB_200_2011.tgz dataset downloaded from the following URL. https://data.caltech.edu/records/65de6-vp158/files/CUB_200_2011.tgz?download=1

Link to my GitHub Repo [here](#)

Model Architecture

This model uses the base layers from a pretrained model called EfficientNetV2. GitHub Repo [here](#)

- Input Layer with input shape(224,224,3)
- A Custom Augmentation Layer, to Augment the images
- Lambda Layer, preprocess the input images using the preprocess_input function of MobileNetV2
- Base Model using the base layers of the pre-trained MobileNetV2 model
- Global Average Pooling, Convert CNN layers to fully connected layer, It computes the average of all pixels in each feature map and returns that as a single element
- Dense layer with 1024 units, 'ReLU' activation
- Dropout Layer
- Dense Layer with 512 units, 'ReLU' activation
- Dropout Layer
- Dense Layer with 200 units, 'softmax' activation

```
model = models.Sequential([
    layers.Input(shape=input_shape),
    CustomAugment(),
    layers.Lambda(preprocess_input),
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(1024, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(num_classes, activation='softmax')
])
```

Model: "sequential"

Layer (type)	Output Shape	Param #
custom_augment (CustomAugment)	(None, 224, 224, 3)	0
lambda (Lambda)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dense (Dense)	(None, 1024)	1,311,744
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524,800
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 200)	102,600

Total params: 4,197,128 (16.01 MB)

Trainable params: 1,939,144 (7.40 MB)

Non-trainable params: 2,257,984 (8.61 MB)

Training Details:

Used the suggested train_test_split provided with the dataset.

I applied the bounding boxes provided with the dataset to crop the images to focus on the subject.



Before applying bounding Box



After applying Bounding Box

- Train samples: 5994 Test samples: 5794 Steps per epoch: 188 Validation steps: 182
- Total number of Parameters: 4,197,128 Trainable Parameters: 1,939,144 Non-trainable Parameters: 2,257,984
- Batch Size = 32 Epochs trained = 36, total epochs were 50, early_stopping stopped it at 36 Optimizer used = Adam(lr = 0.001)
- Total Run Time: 673.3s - GPU T4 x2
- 💡 Train Accuracy: 68.20% Test Accuracy: 63.08%

Model Accuracy and Model Loss curves:

